

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a65467786fb4a3469.jsonl`  
- SHA-256: `7e779a81da87795a88d669f198044be3db9726f3f23a7c4565202a843b19def0`  
- Source modified: `2026-05-31T04:16:19+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:15:26.739Z)

Decompose this research question into complementary search angles.

Question

Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly.

Task

Generate 5 distinct web search queries that together cover the question from different angles. Pick angles that suit the question's domain. Examples:

- broad/primary · academic/technical · recent news · contrarian/skeptical · practitioner/implementation
- For medical: anatomy · common causes · serious differentials · authoritative refs · red flags
- For tech: state-of-art · benchmarks · limitations · industry adoption · cost/tradeoffs

Make queries specific enough to surface high-signal results. Avoid redundancy.

Return: the question (verbatim or lightly normalized), a 1-2 sentence decomposition strategy, and the angles.

Structured output only.

2. assistant (2026-05-31T04:15:29.595Z)

I'll decompose this research question into complementary search angles. Let me first quickly check the existing project context to ground the queries in muneem's actual stack.

3. user (2026-05-31T04:15:30.565Z)

```
1      ---
2      name: hdfc-parser-tech-decision
3      description: Chosen approach + library/license findings for parsing HDFC/bank statement
PDFs
4      metadata:
5          node_type: memory
6          type: project
7          originSessionId: a9fd4426-f234-4e4a-bfb4-1f2c8255c661
8      ---
9
10     Decision (2026-05-31, from a deep-research run): parse HDFC (and other bank) statement
PDFs by **hand-rolling coordinate/column clustering on pdfplumber** ? not by adopting a ready-
made library.
11
12     **Why:** pdfplumber's whole chain is permissive (pdfplumber MIT, pdfminer.six MIT,
Pillow HPND, pypdfium2 Apache/BSD ? no copyleft), it's already a muneem dep, and it opens
password PDFs directly (`pdfplumber.open(path, password=...)`). A prototype word-clustering pass
already recovered ~39/34 of the ~43/38 rows; columns sit at stable x-positions (date ~52,
narration ~111, ref/value-dt ~327?342, withdrawal/deposit ~412?428, balance ~519).
13
14     **Fallback (validated, license-safe):** `camelot-py` (MIT). The old Ghostscript-AGPL
blocker is **gone** ? since v1.0.0 it defaults to pypdfium2 (Apache/BSD); Ghostscript is an opt-
```

```
in `[ghostscript]` extra (do NOT install it). Add Camelot only if pdfplumber clustering proves
insufficient on ruled pages.
15
16     **Do NOT adopt as deps:** `statementsparser` (MIT, multi-bank incl. HDFC) pulls **AGPL
PyMuPDF** mandatorily ? banned; also unproven v0.1.0. `xaneem/hdfc-credit-card-statement-parser`
is MIT + pdfplumber-only but parses **credit-card** statements (not savings/txn) and pins stale
pdfplumber 0.5.28 ? use only as a clustering *reference*.
17
18     **Other findings:** the "convert to Excel/CSV first" idea does NOT improve reliability
(same engine underneath; export is just for QA). `tabula-py` is deprioritized (needs a JVM).
`pypdf` (BSD, already a dep) or `pikepdf` (MPL-2.0, fallback-only) can decrypt-first if ever
needed. No tool was benchmarked on real HDFC savings statements ? always validate against the
user's own files. See [[commercial-license-safety]].
19
```

4. user (2026-05-31T04:15:31.043Z)

```
1     ---
2     name: commercial-license-safety
3     description: "muneem principle ? only permissive, commercially-safe dep licenses
(MIT/BSD/Apache); avoid (A)GPL"
4     metadata:
5         node_type: memory
6         type: feedback
7         originSessionId: a9fd4426-f234-4e4a-bfb4-1f2c8255c661
8     ---
9
10     muneem standing principle (Avi, 2026-05-30): **the project must be safe to use
commercially**, so only add dependencies under **permissive, commercial-safe licenses** ? MIT,
BSD (2/3-clause), Apache-2.0, ISC, HPND, and similar. **Avoid copyleft** (GPL, LGPL, AGPL) and
source-available / non-commercial licenses. MPL-2.0 (weak, file-level copyleft) is tolerable
only as a fallback when no permissive option exists, and must be flagged.
11
12     **Why:** keep muneem free to use and distribute commercially without license
contamination or a required paid commercial license.
13
14     **How to apply:** check a dependency's license **before** adding it. **PyMuPDF / `fitz`
is AGPL-3.0 ? banned** (replaced with pdfminer.six (MIT) for text; pypdfium2 (Apache/BSD) for
page rendering when OCR needs it). Known-good permissive PDF/OCR stack: pdfminer.six (MIT),
pdfplumber (MIT), pypdf (BSD), pypdfium2 (Apache/BSD), Tesseract + pytesseract (Apache-2.0),
scikit-image / numpy (BSD), Pillow (HPND); pikepdf (MPL-2.0) fallback only.
`docs/DEPENDENCIES.md` is the allowlist and `tests/test_dependency_licenses.py` fails the build
if a dependency isn't on it. See [[prefer-local-over-paid-apis]].
15
```

5. assistant (2026-05-31T04:15:43.201Z)

These notes confirm the constraints. I have enough context to decompose this into complementary search angles covering: open VLMs (with the critical per-size license nuances), OCR/table-structure pipelines (with the GPL/non-commercial flags), the borderless-table verification question, and the approval-gated cloud fallbacks.

6. user (2026-05-31T04:16:11.394Z)

Structured output provided successfully

7. assistant (2026-05-31T04:16:19.379Z)

I've decomposed the research question into 6 complementary search angles, returned via the StructuredOutput tool.

The decomposition splits across the two model families the question centers on (license-gated open VLMs vs. OCR/table-structure pipelines), then carves out dedicated angles for the highest-risk decision drivers: the per-size/weights license minefield (the constraint that disqualifies

most candidates), the borderless-table accuracy + financial-number hallucination/verification question, OCR-specialist VLMs with India-specific bank-statement evidence, and the approval-gated cloud fallbacks with their data-handling terms.

Two project files informed the angles (both confirming the license-safety framing rather than altering it):

- `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-muneem\memory\hdfc-parser-tech-decision.md` ? current decision is hand-rolled coordinate clustering on pdfplumber (MIT chain), with camelot-py (MIT) as the validated fallback; no VLM/OCR tier chosen yet, which is what this research informs.
- `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-muneem\memory\commercial-license-safety.md` ? only MIT/BSD/Apache/ISC/HPND deps allowed; (A)GPL and non-commercial banned (PyMuPDF already banned), MPL-2.0 tolerable only as flagged fallback. This is why the weights-license angle is broken out as its own decisive query.